



Apache Kafka: Conceptos Básicos para Devs

Del log distribuido a la arquitectura event-driven

Kafka 4.x · KRaft · Sin ZooKeeper

Kafka en una oración



Alta disponibilidad
Replicación automática
entre brokers.



Persistencia
Retención configurable
(días, semanas, meses)

**Un log distribuido,
particionado y
replicado al que
múltiples productores
escriben y múltiples
consumidores leen a
su propio ritmo.**

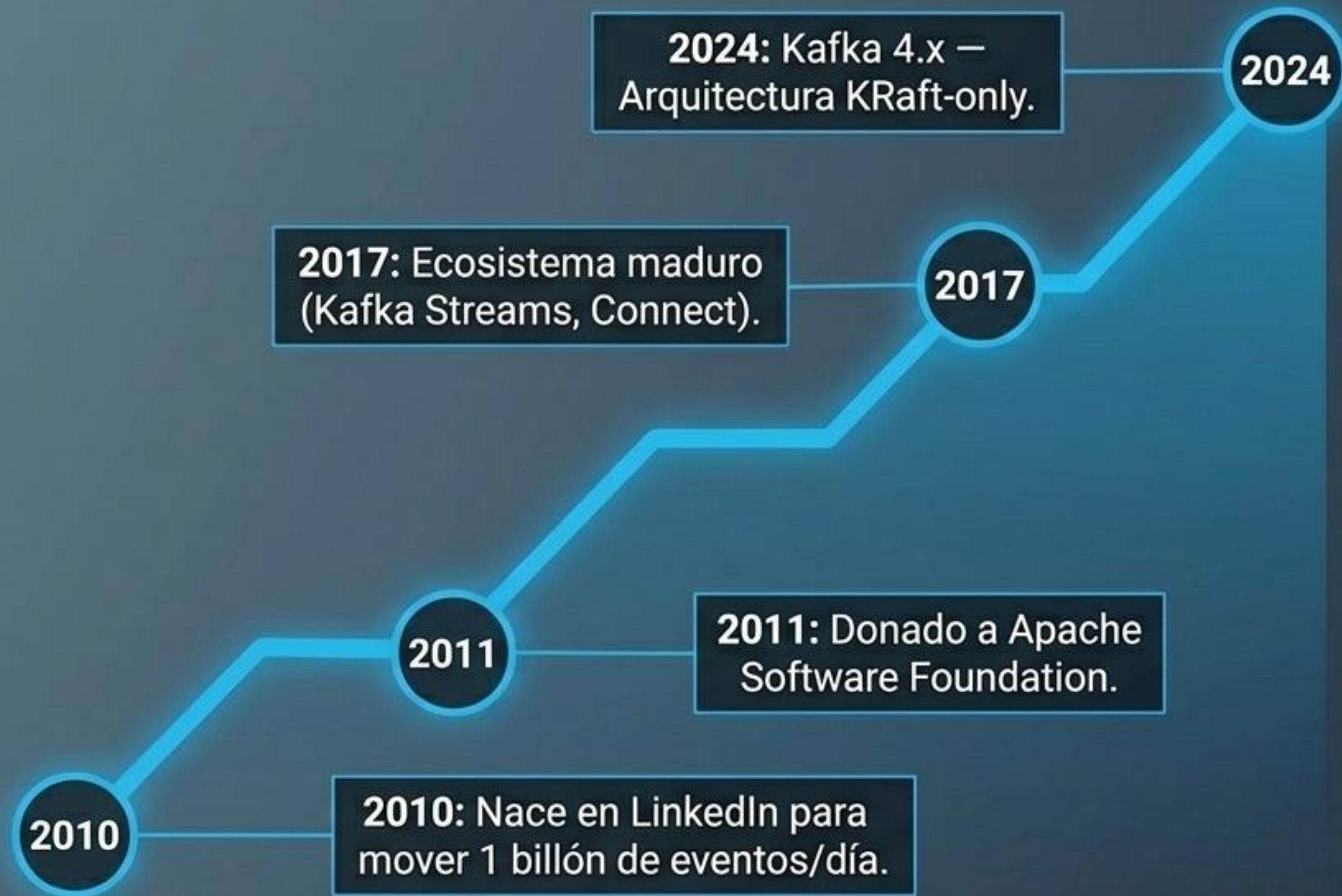


Alto rendimiento
Millones de eventos por
segundo con latencia de
milisegundos.



Replay
Reprocesamiento de
eventos históricos sin
afectar a los productores

De LinkedIn al estándar de la industria



El impacto de KRaft

Adiós ZooKeeper. Arranque 10x más rápido, operación simplificada radicalmente, soporte para millones de particiones.

Dato de impacto

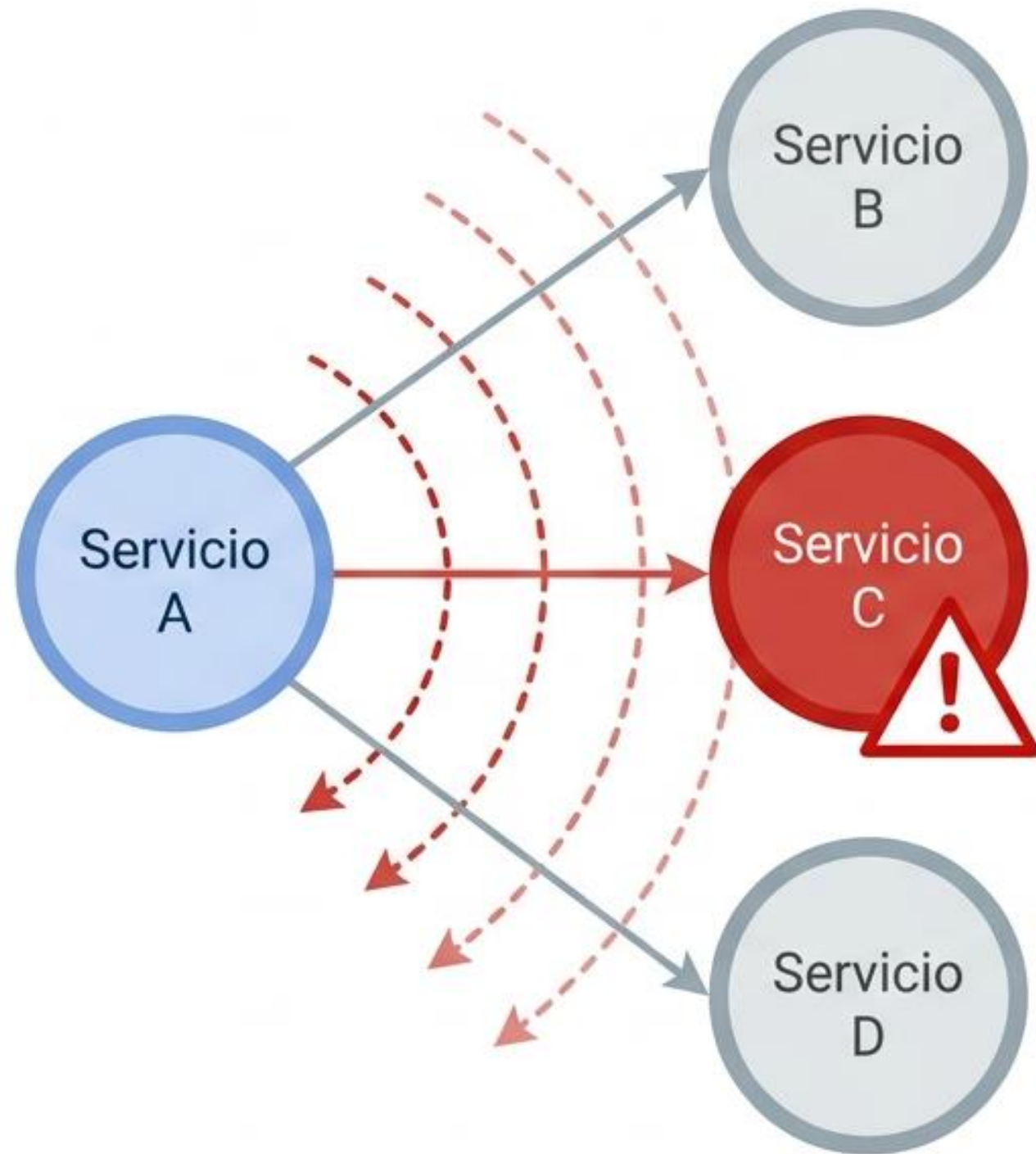
+80%
de empresas
Fortune 100 lo usan
en producción.

30 años de mensajería: hacia el log distribuido

1990s	2000s	2011+
Paradigma: Message Brokers (Colas)	Paradigma: ESB / SOA	Paradigma: El Log Distribuido
Ejemplos: ActiveMQ, RabbitMQ	Ejemplos: Oracle Service Bus	Herramienta: Apache Kafka
Limitación: Punto a punto, lectura destructiva	Limitación: Monolito de integración, cuellos de botella	Solución: Persistencia, escalabilidad horizontal, replay

**En una cola tradicional, leer es borrar. En Kafka, leer es consultar.
Ese es el cambio de paradigma.**

Cuando el REST síncrono duele



El usuario **espera 10 segundos** aunque otros servicios respondan al instante. El sistema es tan rápido como su **eslabón más débil**.

1 servicio caído =
operación completa
bloqueada

Escalar implica escalar
todos los nodos
interdependientes

Sin historial de
eventos (debugging
ciego en producción)

Cada integración
nueva requiere
código custom

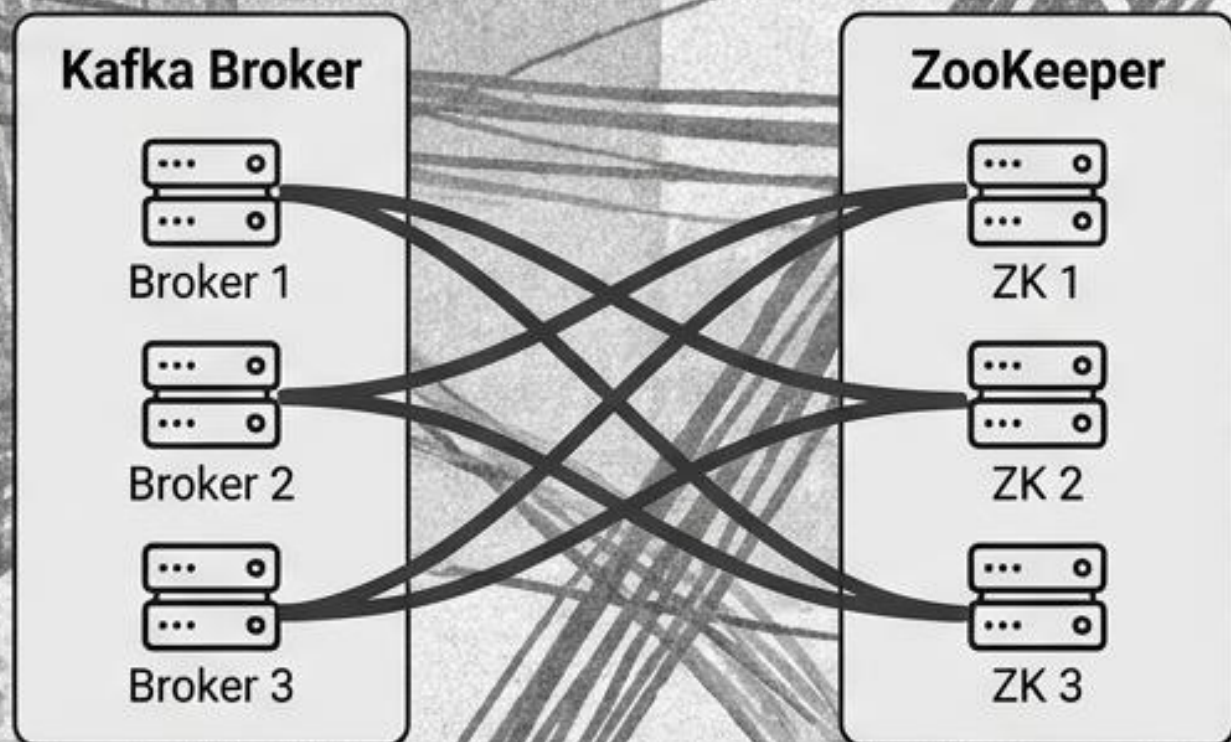
Arquitectura general de alto nivel



Las 3 piezas del motor interno: Topics, Particiones y Offsets.

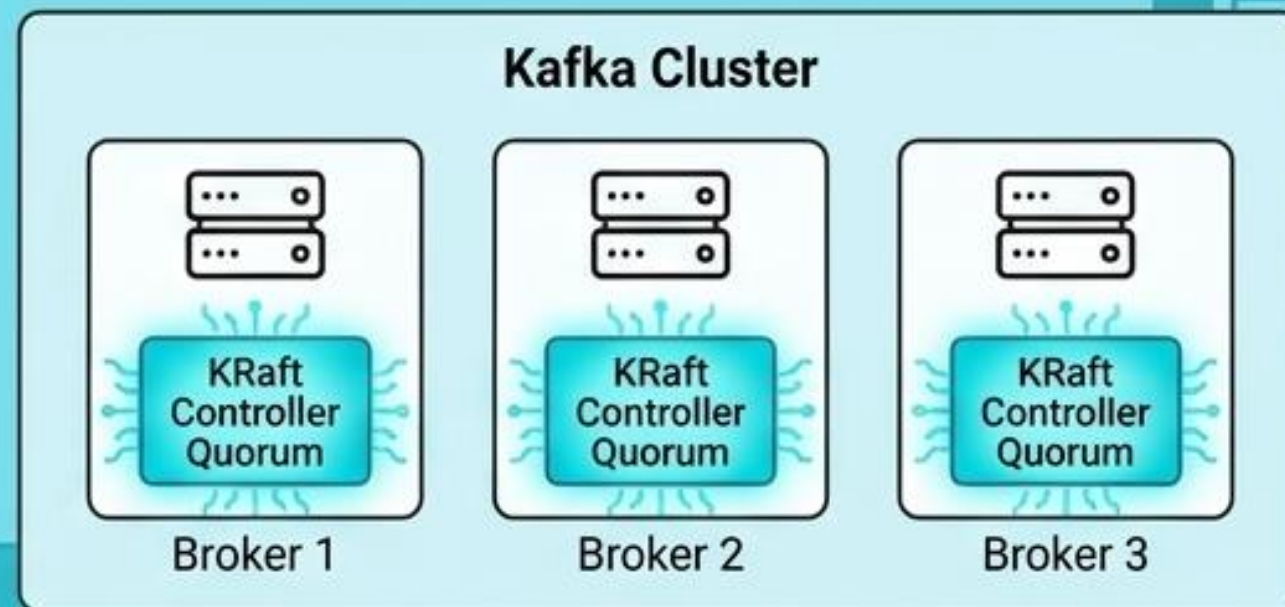
KRaft elimina ZooKeeper y simplifica la arquitectura en Kafka 4.x

Antes (Kafka < 3.x)



Dos sistemas distribuidos independientes que instalar, asegurar y operar. Límite práctico de ~200,000 particiones.

Ahora (Kafka 4.x con KRaft)



El protocolo de consenso (Raft) vive directamente dentro de los brokers. Cero dependencias externas.



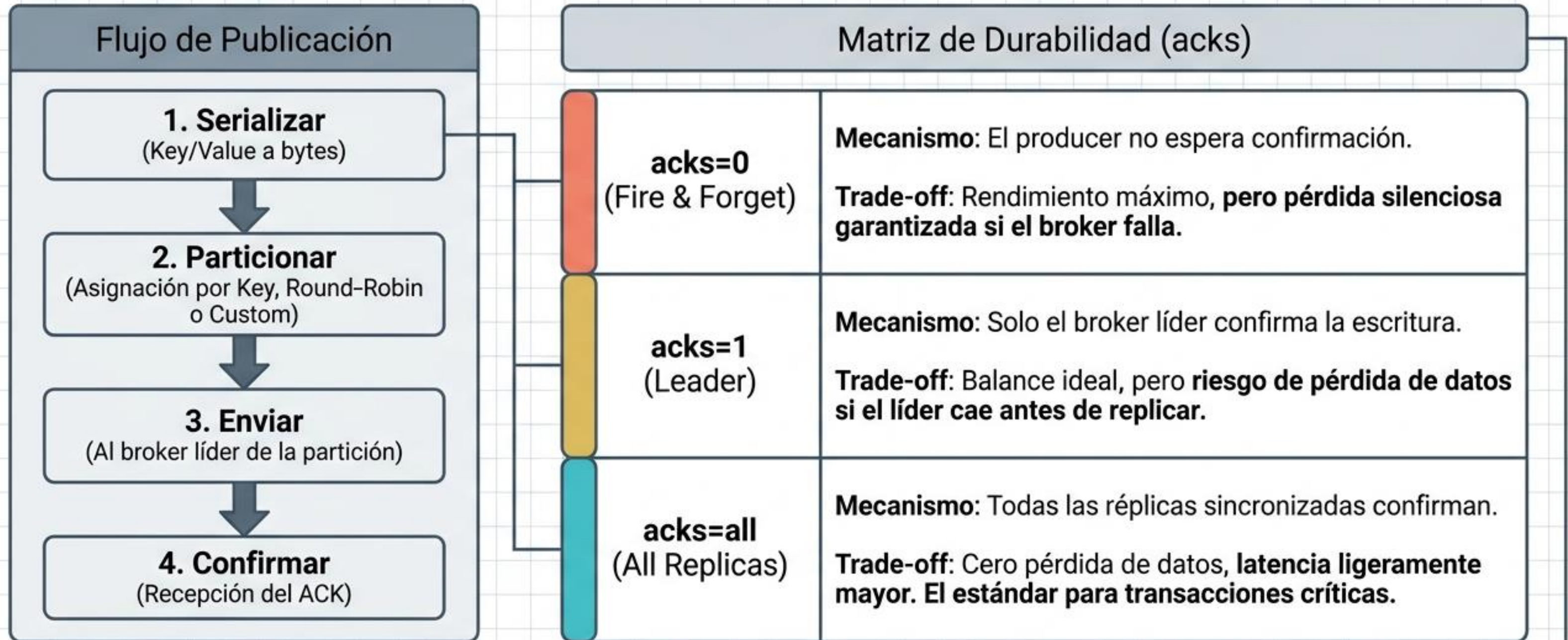
Beneficios Clave & Dev Experience

1. Arranque y apagado 10x más rápido. Soporta millones de particiones.

```
docker run apache/kafka:4.2.0
```

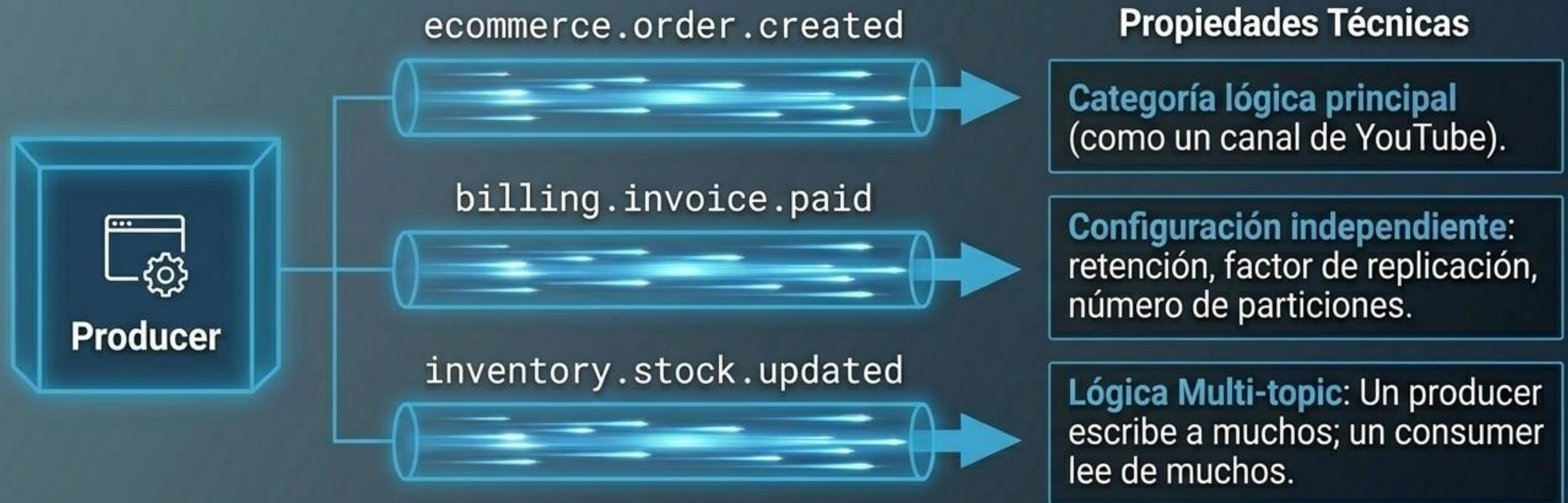
Experiencia Local: Basta un solo contenedor para empezar a desarrollar.

Producers publican los datos y controlan la durabilidad



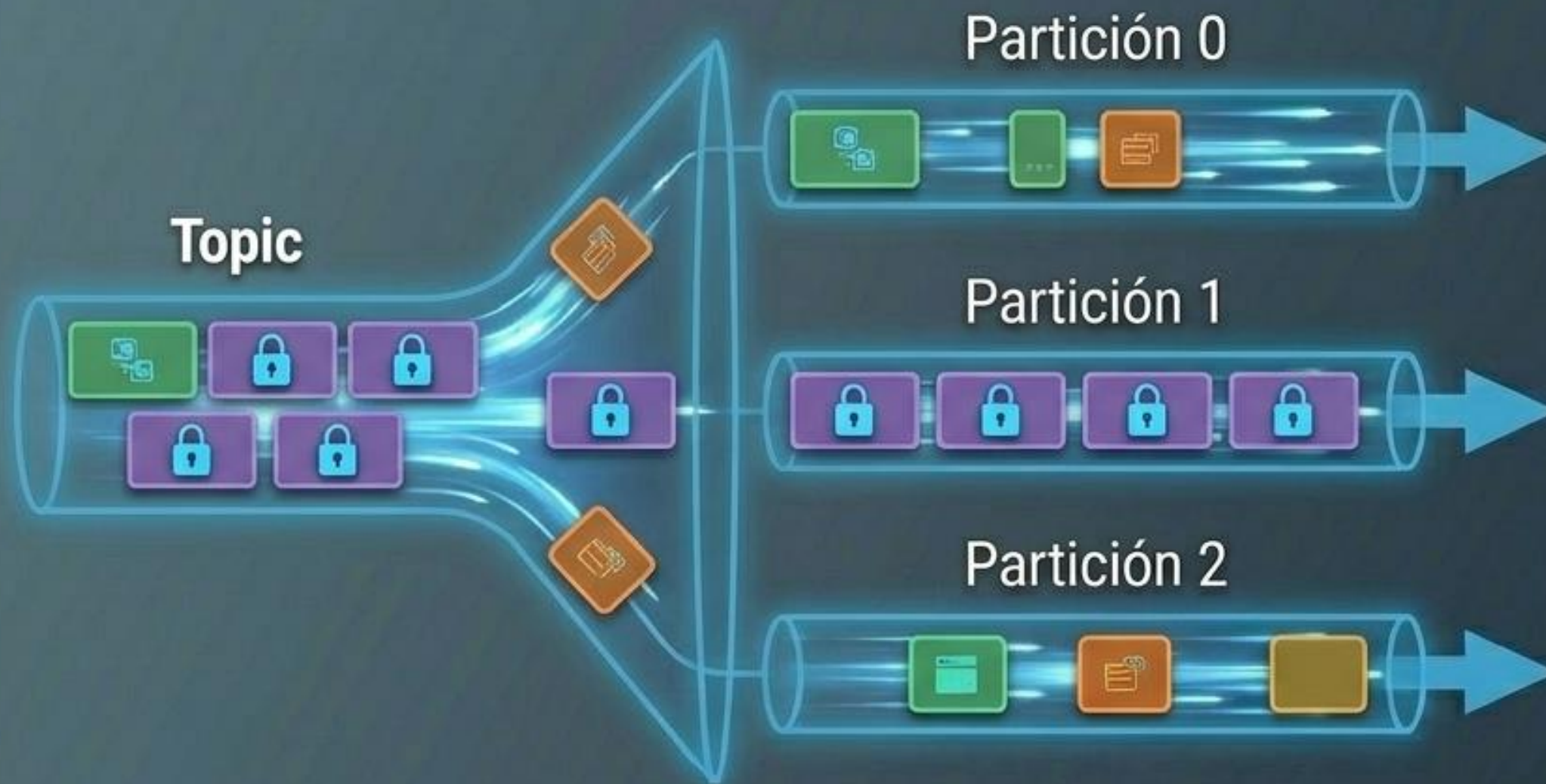
Exactly-once delivery es posible combinando **acks=all** con la configuración **enable.idempotence=true**, garantizando que ningún reintento genere mensajes duplicados.

Topic: El canal con nombre



Convención recomendada: `<dominio>.<entidad>.<evento>` (ej. `users.profile.updated`).

Particiones: Cómo escala Kafka



Reglas de Oro

1. Escalabilidad horizontal máxima: más particiones = más paralelismo.
2. **El orden solo se garantiza DENTRO de una partición.**
3. La clave de particionado (Partition Key) determina la ruta de ruteo exacta.



Si necesitas orden global → usa **1 sola partición**.
Si necesitas escala → usa **múltiples particiones y una clave**.

Offset: El puntero inmutable

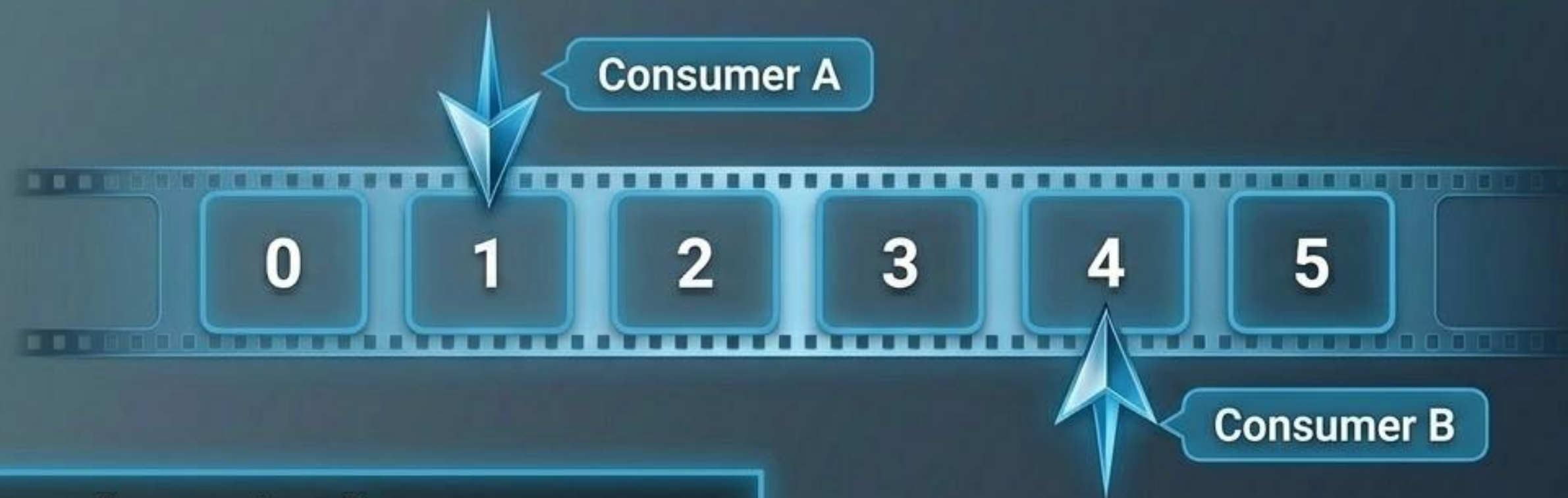


Definiciones

- El puntero exacto e inmutable de cada mensaje.
- Cada Consumer Group mantiene sus propios offsets.

El **Consumer Lag** es tu métrica reina. Si el lag sube constantemente, tu consumer no da abasto con la velocidad de producción de datos.

La idea que lo cambia todo: El Log Distribuido



Conceptos Core

Inmutabilidad: Los mensajes no se modifican ni se borran al ser leídos.

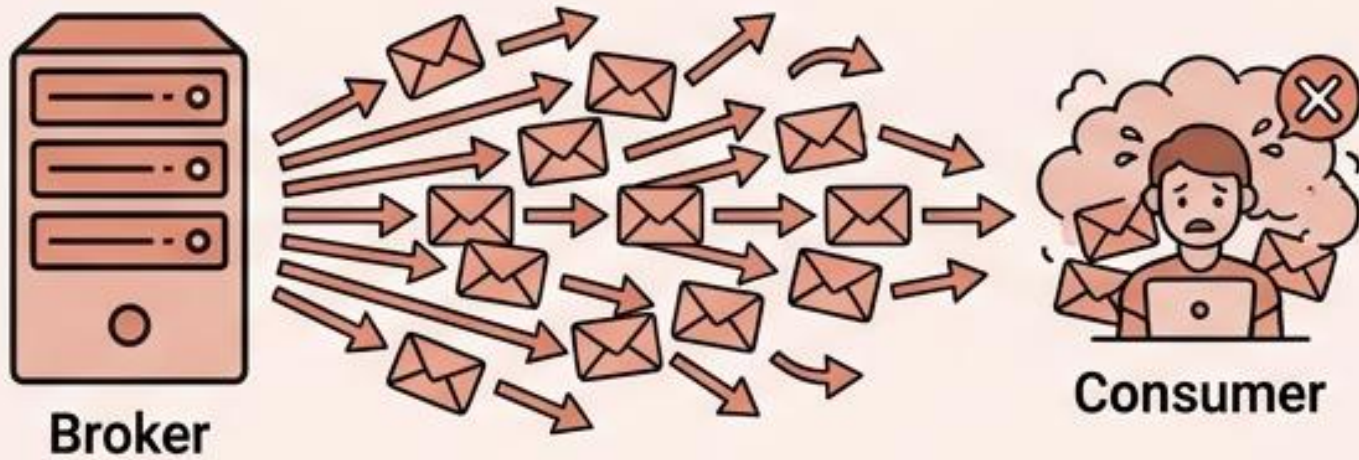
Orden estricto: Secuencia garantizada e inalterable.

Independencia: Múltiples consumers leen el mismo log de forma totalmente aislada.

“Un nuevo servicio puede conectarse hoy y leer la historia desde el inicio (replay) sin afectar a nadie más.”

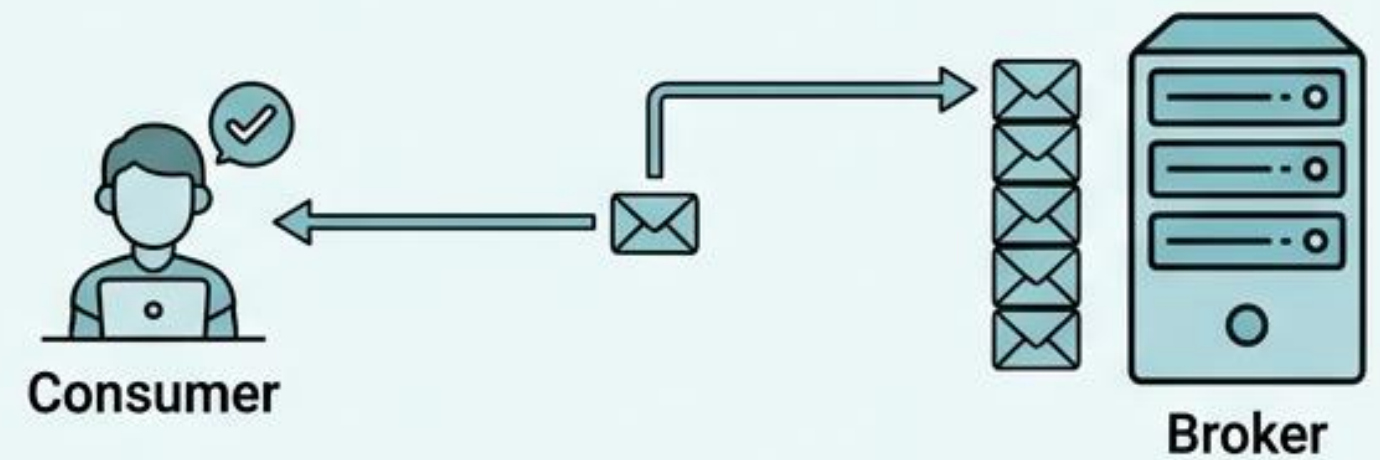
Los consumers controlan su ritmo mediante el modelo Pull

Modelo **Push** (Colas Tradicionales)



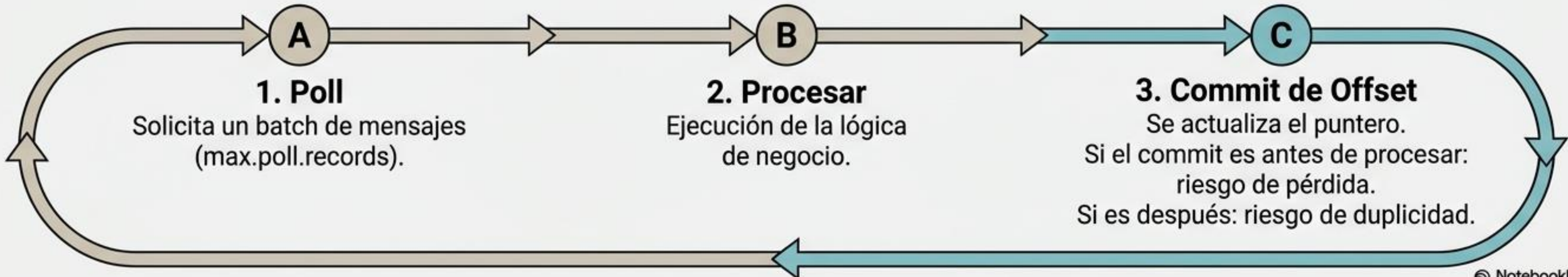
El broker fuerza los mensajes hacia el consumer.
Alto riesgo de saturación (falta de back-pressure).

Modelo **Pull** (Apache Kafka)

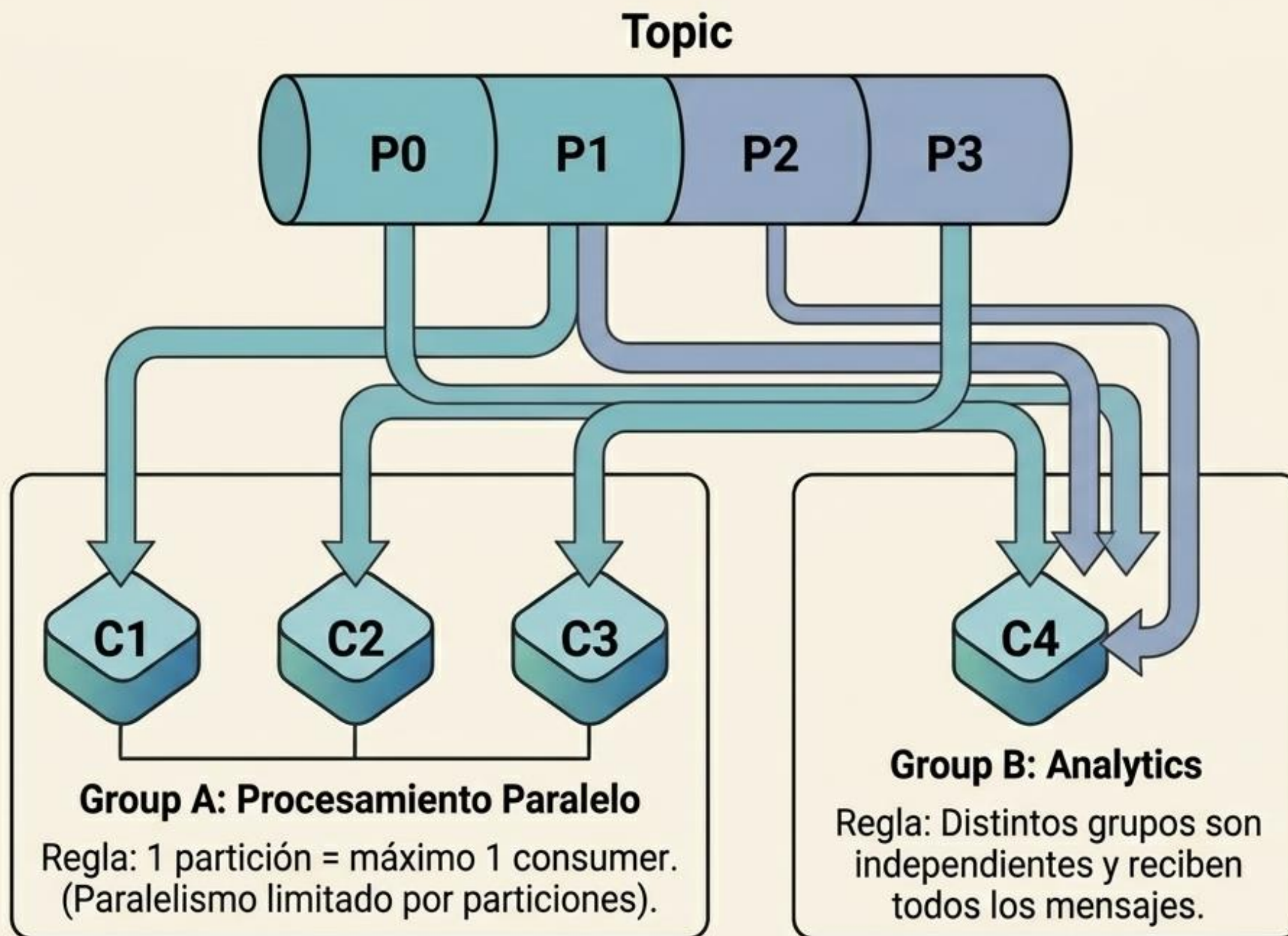


El consumer jala (pulls) los datos cuando está listo.
Control total sobre la velocidad de procesamiento.

Ciclo de Vida del Consumer



Consumer Groups habilitan el procesamiento en paralelo



Rebalanceo Automático



Fallo

Consumer 2 muere por un fallo.



2. Detección

El clúster detecta la caída por timeout del heartbeat.

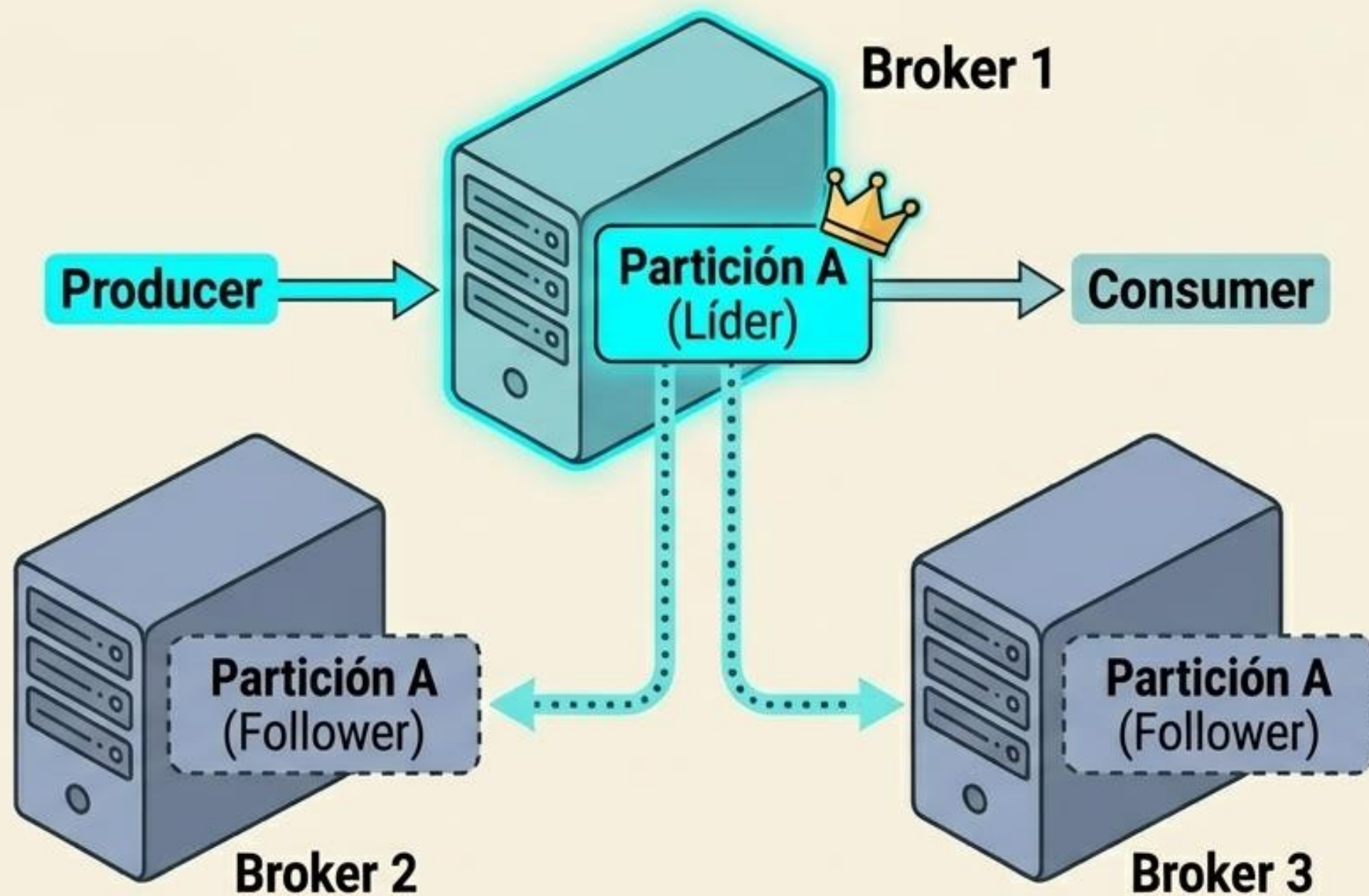


3. Rebalanceo

La partición 2 se reasigna automáticamente a Consumer 1 o 3 en segundos, sin intervención.

Brokers y clústeres garantizan la tolerancia a fallos

El Corazón Distribuido



Si el Broker 1 cae: Nuevo líder elegido automáticamente en milisegundos.

Brokers

Los servidores físicos o virtuales que almacenan las particiones y atienden peticiones.

Líder vs. Follower

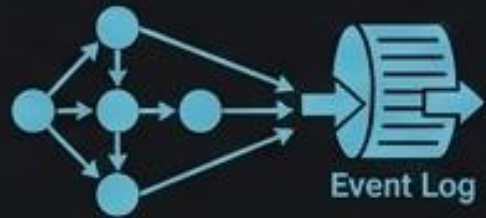
Toda la lectura y escritura ocurre en el Líder. Los Followers son copias de seguridad pasivas.

Factor de Replicación (RF)

El estándar de producción es RF=3. Garantiza que el clúster puede sobrevivir a la pérdida total de un broker sin perder un solo byte de datos.

Patrones reales de Kafka en producción

¿Dónde vive Kafka en el mundo real?



Event Sourcing

Microservicios comunicándose vía eventos asíncronos en lugar de llamadas REST directas.

Ej: Uber, Airbnb



Pipelines de Datos

Columna vertebral para mover datos en tiempo real entre sistemas transaccionales y analíticos.

Ej: LinkedIn



Telemetría en Tiempo Real

Ingesta masiva de logs, métricas y trazas de miles de servidores concurrentes.

Ej: Netflix, Cloudflare



Detección de Fraude

Análisis de streams de transacciones financieras correlacionadas en milisegundos.

Ej: Visa, PayPal



Procesamiento IoT

Ingesta simultánea de telemetría de millones de dispositivos y sensores en el borde.

Ej: Tesla, GE



Change Data Capture (CDC)

Captura de cambios (inserts/updates) en bases de datos relacionales usando Debezium.

Ej: Ecosistemas de bases de datos

Cuándo elegir Kafka frente a otras alternativas de mensajería

[Dimensiones]	Apache Kafka	RabbitMQ	Amazon SQS	Redis Pub/Sub
Modelo de Datos	Log Distribuido	Cola de Mensajes	Cola Gestionada	Pub/Sub en memoria
Retención	Configurable (Días/Semanas)	Hasta que se consume	Hasta 14 días	No existe (Fire & forget)
Replay (Reprocesamiento)	Sí (Historial completo)	No	No	No
Throughput (Escala)	Millones/seg	Miles/seg	Miles/seg	Millones/seg (sin durabilidad)
Caso de Uso Ideal	Streaming de eventos y alta escala	Tareas y workflows simples	Desacoplamiento serverless AWS	Notificaciones efímeras RT

Cuándo NO usar Kafka: Si tu sistema maneja un bajo volumen de mensajes, no requiere procesamiento histórico, o busca encolar trabajos simples (task queues), herramientas como SQS o RabbitMQ reducirán significativamente la complejidad operativa.

Lo que llevas hoy (Síntesis)

Los 6 Conceptos Fundamentales

1. Topic

El canal lógico. Es la categoría con nombre hacia donde fluyen los eventos del negocio.

2. Partición

La unidad de escalado físico. Aquí reside el orden garantizado y el paralelismo real.

3. Offset

El puntero inmutable. Marca la posición exacta de cada mensaje en el historial del log.

4. Producer

Quien escribe los datos. Toma la decisión crucial entre rendimiento vs. durabilidad (acks).

5. Consumer

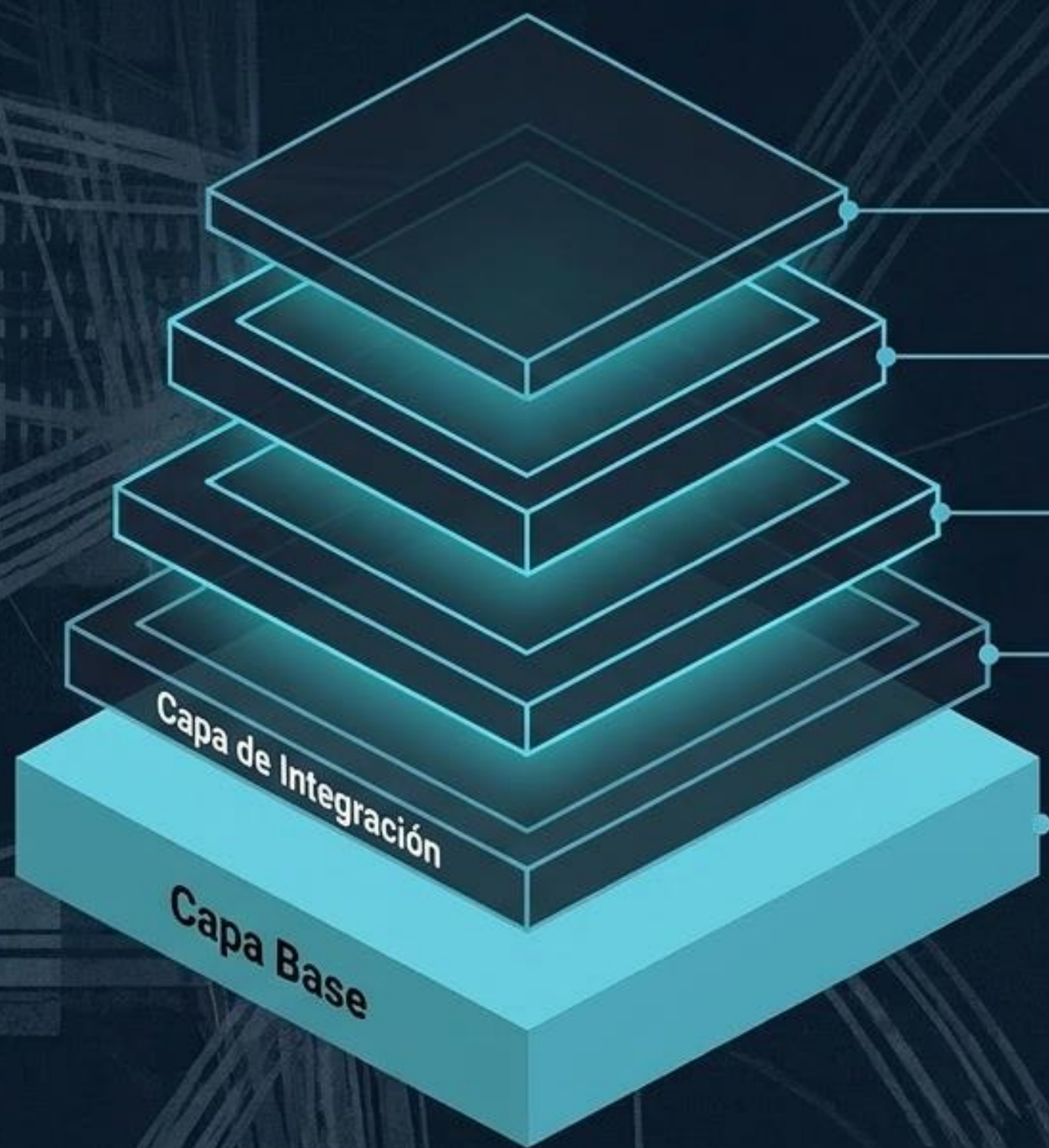
Quien lee los datos. Opera mediante modelo pull, avanzando a su propio ritmo sin saturarse.

6. Consumer Group

El escuadrón de procesamiento. Se reparten las particiones para lograr escalado horizontal dinámico.

Kafka no es una cola. Es la memoria distribuida de tu sistema.

El ecosistema expande a Kafka de un log a una plataforma



ksqlDB

Base de datos de streaming que permite ejecutar consultas continuas sobre los eventos utilizando sintaxis SQL familiar.

Schema Registry

Gestión centralizada de contratos de datos. Valida formatos (Avro, Protobuf) en tiempo real entre producers y consumers.

Kafka Streams & Apache Flink

Motores para procesar, transformar, filtrar y hacer joins de streams de datos en tiempo real directamente en memoria.

Kafka Connect

Framework no-code para integrar más de 200 fuentes externas (Source/Sink conectores para DBs, S3, Elastic).

Apache Kafka Core

Brokers, Topics, Particiones, KRaft. El motor de almacenamiento inmutable y distribución de eventos.

Hoja de ruta táctica para dominar la plataforma

Próximos pasos



Día 1: Manos en la masa

- ✓ Ejecutar KRaft en un solo contenedor (**docker run**).
- ✓ Crear tu primer Topic utilizando la CLI oficial.
- ✓ Publicar y leer mensajes básicos usando los scripts console-producer y console-consumer.



Semana 1: Código y Clientes

- ✓ Integrar el cliente oficial en tu lenguaje (Java, Python, Go).
- ✓ Implementar **commitSync** y **commitAsync** manuales en un consumer.
- ✓ Levantar múltiples instancias para visualizar el rebalanceo automático en Consumer Groups.



Mes 1: Operaciones y Ecosistema

- ✓ Diseñar estrategias de particionado utilizando Claves de negocio (Keys).
- ✓ Configurar un conector de Kafka Connect para ingestar datos.
- ✓ Monitorear la métrica más crítica de producción: el Consumer Lag.

